

A Neural Network for the Identical Kuramoto Equation: Architectural Considerations and Performance Evaluation

1st Nishantak Panigrahi

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology (VIT)
Vellore, India
nishantak09@gmail.com,
nishantak.panigrahi2022@vitstudent.ac.in

2nd Mayank Patwal

School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology (VIT)
Vellore, India
mayanksingh4370@gmail.com,
mayanksingh.patwal2022@vitstudent.ac.in

Abstract—In this paper, we investigate the efficiency of Deep Neural Networks (DNNs) to approximate the solution of a nonlocal conservation law derived from the identical-oscillator Kuramoto model, focusing on the evaluation of an architectural choice and its impact on solution accuracy based on the energy norm and computation time. Through systematic experimentation, we demonstrate that network configuration parameters—specifically, activation function selection (tanh vs. sin vs. ReLU), network depth (4-8 hidden layers), width (64-256 neurons), and training methodology (collocation points, epoch count)—significantly influence convergence characteristics. We observe that tanh activation yields stable convergence across configurations, whereas sine activation can attain marginally lower errors and training times in isolated cases, but occasionally produce nonphysical artefacts. Our comparative analysis with traditional numerical methods shows that optimally configured DNNs offer competitive accuracy with notably different computational trade-offs. Furthermore, we identify fundamental limitations of standard feed-forward architectures when handling singular or piecewise-constant solutions, providing empirical evidence that such networks inherently oversmooth sharp features due to the natural function space limitations of standard activation functions. This work contributes to the growing body of research on neural network-based scientific computing by providing practitioners with empirical guidelines for DNN implementation while illuminating fundamental theoretical constraints that must be overcome to expand their applicability to more challenging physical systems with discontinuities.

Index Terms—Deep Neural Network Architecture, Activation Functions, Physics-Informed Neural Network, Hyperparameter Tuning, Scientific Computing, Nonlocal Conservation Law, Kuramoto Model, Convergence Analysis, Energy Norm

I. INTRODUCTION

Partial Differential Equations (PDEs) are ubiquitous in modelling processes and their properties arising from a vast range of diverse fields, such as real-time engineering problems, different physical phenomena, chemical and biological events, and socio-economic theory [1] [2]. Efficient and accurate solution of such complex dynamical systems becomes critical across a wide range of these applications. Classical numerical methods, such as finite-volume schemes can deliver accuracy but at the cost of long runtimes, especially for high-resolution

simulations [3]. Deep neural networks (DNNs) offer a mesh-free alternative by learning a continuous solution directly from the data known about the process, for example, the PDE. We enhance the DNN by embedding the PDE residual and boundary/initial-condition constraints into the network’s loss function, a paradigm henceforth referred to as a Physics-Informed Neural Network (PINN) [4]. Consequently, we obtain a model that, once trained, produces solutions to the PDE almost instantaneously. Furthermore, PINNs generalise better beyond the training domain and often exhibit faster training rates than traditional DNNs due to their incorporation of the governing physical laws directly into the loss function, thereby enforcing strong regularisation and consistency with known physics, and reducing reliance on large datasets [5]. This makes PINNs particularly attractive for rapid computation tasks in mission-critical workflows.

Despite these advances, the use of PINNs for nonlocal conservation laws that arise in a large variety of applications, particularly those derived from the Kuramoto model, remains underexplored. Moreover, deploying PINNs in real-world engineering requires a clear understanding of how architectural choices (e.g., activation functions, network depth/width) and training strategies (e.g., collocation sampling, epoch budgets) affect the solution fidelity.

In this paper, we fill this gap by conducting a comprehensive evaluation of DNN architectures by approximating the solution to the identical-oscillator Kuramoto Equation. Our contributions are threefold:

- **Architectural analysis:** We compare tanh, sin, and ReLU activations across network depths (4-8 layers) and widths (64-256 neurons), quantifying their influence on convergence and error. Highlighting cases where sine activations can yield marginal error reductions at the risk of nonphysical oscillations, and showcasing the failure of ReLU activation in capturing the evolution of the Kuramoto model.
- **Hyperparameter study:** We sweep collocation-point sampling and epoch count, to identify configurations

that optimise the trade-off between accuracy and training time.

- **Limitations and guidelines:** We demonstrate intrinsic oversmoothing of discontinuities by standard feed-forward DNNs and provide practical recommendations and empirical guidelines for practitioners.

II. PHYSICS-INFORMED NEURAL NETWORK FORMULATION

A. Identical Kuramoto Equation

We approximate the solution to the identical Kuramoto equation described in [6], by approximating the phase density $u(\theta, t)$, satisfying the PDE

$$\frac{\partial u}{\partial t}(\theta, t) + \frac{\partial}{\partial \theta} [V[u](\theta, t) u(\theta, t)] = 0, \quad \theta \in [0, 2\pi], \quad t \in [0, T] \quad (1)$$

with smooth, polynomial initial condition described in [7] as

$$u(\theta, 0) = u_0(\theta), \quad \theta \in [0, 2\pi].$$

$$u_0(\theta) = \begin{cases} \frac{6}{\pi^3} \left(\frac{3\pi}{2} - \theta \right) \left(\theta - \frac{\pi}{2} \right), & \text{if } \theta \in \left[\frac{\pi}{2}, \frac{3\pi}{2} \right], \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

Where, the nonlocal flux is given by the convolution

$$V[u](\theta, t) = -K \int_0^{2\pi} \sin(\theta - \phi) u(\phi, t) d\phi \quad (3)$$

For our simulation we have set the final time, $T = 1$.

B. Network Architecture

A Deep Neural Network (DNN) is a parameterised function $u_\Phi : \mathbb{R}^d \rightarrow \mathbb{R}$, defined as a composition of affine maps and nonlinear activations. Effectively, our PINN learns the solution to the identical Kuramoto equation by a mapping from input $x = (\theta, t) \in \mathbb{R}^2$, to an output $u(\theta, t) \in \mathbb{R}$.

The PINN, $\mathcal{N}_\Phi$, is constructed as a fully-connected feed-forward network, as shown in Fig. 1, with an input layer of dimension 2 (receiving spatial and temporal coordinates), L hidden layers, each with n neurons, with nonlinear activation functions σ , and a linear output layer that yields the approximation $u(\theta, t)$,

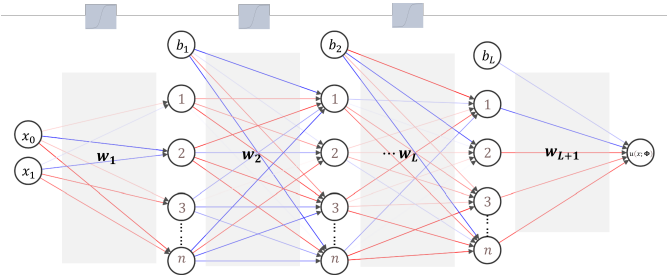


Fig. 1: Deep neural network architecture.

It is formulated as

$$\mathcal{N}(x; \Phi) : \mathbb{R}^2 \rightarrow \mathbb{R}$$

For our architecture we have chosen $L=4$, $n=64$, and $\sigma = \tanh$. This gives us

$$\begin{aligned} x &= [\theta, t]^\top, \\ \mathbf{h}^{(0)} &= x, \\ \mathbf{h}^{(i)} &= \tanh(W^{(i)} \mathbf{h}^{(i-1)} + \mathbf{b}^{(i)}), \quad i = 1, \dots, 4, \\ \mathcal{N}(x; \Phi) &= W^5 \mathbf{h}^{(4)} + \mathbf{b}^{(5)} = u(\theta, t) \end{aligned}$$

The trainable parameters, $\Phi = \{W^{(i)}, \mathbf{b}^{(i)}\}_{i=1}^5$, are learnt by a minimising a scalar loss $L(\Phi)$.

Where for layer i with n_i neurons,

- $W^{(i)} \in \mathbb{R}^{n_i \times n_{i-1}}$ is the weight matrix,
- $\mathbf{b}^{(i)} \in \mathbb{R}^{n_i}$ is bias matrix,

C. Loss Components

1) *PDE Residual*: Defines the point-wise residual of our PDE

$$r(\theta, t; \Phi) = \frac{\partial u_\Phi}{\partial t} + \frac{\partial}{\partial \theta} (V[u_\Phi] u_\Phi) \quad (4)$$

Sampling $N_r=1024$ collocation points (θ_i, t_i) , drawn uniformly via Latin-Hypercube sampling over $[0, 2\pi] \times [0, T]$, we form

$$L_{\text{res}} = \frac{1}{N_r} \sum_{i=1}^{N_r} r(\theta_i, t_i; \Phi)^2 \quad (5)$$

Nonlocal Flux: Nonlocal convolution, $V[u]$ is computed via a Riemann sum. We introduce $N_q=128$ uniformly spaced nodes $\{\phi_j\}_{j=1}^{N_q}$ on $[0, 2\pi]$ with step size $\Delta\phi = 2\pi/N_q$. The integral is then approximated as

$$V[u_\Phi](\theta_i, t_i) \approx -K \sum_{j=1}^{N_q} \sin(\theta_i - \phi_j) u_\Phi(\phi_j, t_i) \Delta\phi \quad (6)$$

2) *Initial Condition Loss*: We enforce $u_\Phi(\theta, 0) = u_0(\theta)$ by sampling $N_0=512$ initial condition points, $\{\theta_j^0\}_{j=1}^{N_0}$ over $[0, 2\pi]$, and computing

$$L_{\text{IC}} = \frac{1}{N_0} \sum_{j=1}^{N_0} |u_\Phi(\theta_j^0, 0) - u_0(\theta_j^0)|^2 \quad (7)$$

3) *Total Loss*: The total physics-informed loss that the network sees becomes a weighted sum

$$L_{\text{total}} = \lambda_{\text{res}} \cdot L_{\text{res}} + \lambda_{\text{IC}} \cdot L_{\text{IC}}.$$

where λ_{res} , λ_{IC} are the weights of the residual loss and the initial condition loss respectively. In our implementation we have set $\lambda_{\text{res}} = \lambda_{\text{IC}} = 1$.

D. Training and Optimisation

- **Automatic Differentiation**: All derivatives $\partial_t u_\Phi$ and $\partial_\theta (\mathbf{F}_\Phi)$ are computed via the framework's auto-grad engine, which is significantly faster than numerical differentiation and one of the motivations behind using neural networks for scientific computing [8].
- **Optimiser**: Model parameters are updated using back-propagation gradients by the Adam optimiser, with learning rate 10^{-3} , typically over 4096 or 5120 epochs.

III. EXPERIMENTAL SETUP & METHODOLOGY

In the following experiments, simulation parameters are purely synthetic, approximating different order of smoothnesses occurring in nature.

A. Network Architecture and Hyperparameter Grid

We train and evaluate fully-connected, feed-forward DNNs by systematically varying the architecture as follows:

- Activation function, $\sigma \in \{\tanh, \text{sine}, \text{ReLU}\}$
- Network depth, L , and width, n ,

$$\{L, n\} \in \{\{4, 64\}, \{4, 128\}, \{6, 128\}, \{6, 256\}, \{8, 256\}\}$$

We vary training hyperparameters as follows:

- Epoch counts, $n_e \in \{2048, 4096, 5120, 10240\}$
- Collocation points, $N_r \in \{1024, 2048\}$

Each trained network, $\mathcal{N}_{\Phi}^{\{\sigma, \{L, n\}, n_e, N_r\}}$, learns the trainable parameters Φ to map an input $x = \{\theta, t\}$ to the output $u_{\Phi}(\theta, t)$, passing it through L hidden layers of size n , and a linear output layer.

B. Evaluation Metrics

1) *Exact Solution*: We compute a high-fidelity solution, $u_{ex}(\theta, t)$, using a high-resolution finite-volume solver described in [7].

A super-fine mesh solution with 512 spatial points, and 205 temporal points over the domain, is saved to serve as the reference solution for evaluating and comparing the accuracy of the PINN solutions. The method uses a finite-volume scheme with Lax-Friedrichs numerical flux.

2) *Energy Norm*: We quantify the solution accuracy by calculating the discrete L^2 norm

$$\|u_{\Phi} - u_{ex}\|_E = \sqrt{\frac{1}{N_{eval}} \sum_{k=1}^{N_{eval}} (u_{\Phi}(\theta_k, t_k) - u_{ex}(\theta_k, t_k))^2} \quad (8)$$

This gives us a single scalar measuring the deviation between the network's prediction and the high-fidelity reference solution. Physically, $\|u_{\Phi} - u_{ex}\|_E$ represents the *total energy* of the error field, accumulated over all phases and times, giving us a metric for direct comparison of architectures.

3) *Training Time*: The total wall-clock time each network configuration takes to finish training is measured in *seconds* using Python's *time* module, and serves as a direct metric of computational efficiency across the grid.

C. Training Hardware

All experiments are conducted on a laptop configured with an AMD Ryzen 7 7735HS CPU, 16 GB of system memory, and an NVIDIA RTX 3050 dGPU (6 GB VRAM), running CUDA 12.8. Neural network training is performed on the dGPU via PyTorch's CUDA backend, with a fixed seed to ensure consistency and reproducibility of timing measurements.

IV. RESULTS AND DISCUSSIONS

In this section, we present a quantitative evaluation of the trained networks across our architectural and hyperparameter grid, and analyse the effect of varying activation function, network depth and width, collocation density, and epoch budget, on solution accuracy and training efficiency.

A. Activation Functions

Networks with hyperbolic-tangent activation, $\tanh$, consistently demonstrate inherently reliable convergence in smooth function appropriators. The best-performing configuration $\sigma = \tanh, \{L=4, n=128\}, N_r=1024, n_e=4096$ showed the least energy norm, $RMSE = 6.65 \times 10^{-5}$ with a training time of ~ 114 *seconds*. The evolution of the solution is shown in Fig. 2. Approximately 5% of $\tanh$ networks showed nonphysical artefacts. Importantly, these occurrences were confined to configurations with excessive depth and width, suggesting that over-parameterisation for a simple PDE leads the model to fit spurious features rather than generalise the true solution structure.

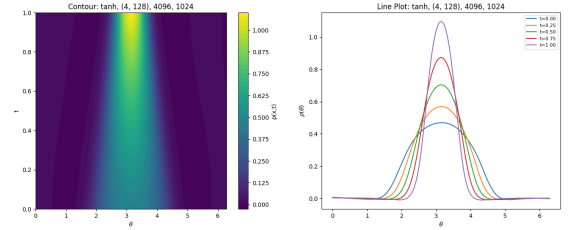


Fig. 2: Simulation profile of tanh network.

Best-case trials for sine networks occasionally yielded marginally lower energy norm for the same architecture with $\tanh$ activation, showing upto 10% lower energy norms, and approximately 13% faster convergence than $\tanh$ architectures with similar energy norm, as shown in Fig. 3. The evolution of the solution is shown in Fig. 4. However, such networks produced nonphysical artefacts, as shown in Fig. 5, in approximately 10% of configurations. These artefacts were, predominantly, spurious high-frequency oscillations which can be attributed to the intrinsic oscillatory nature of the sine activation, whose nondecaying, sign-alternating derivative, cosine, amplifies unresolved spectral modes under insufficient collocation density, a behaviour not observed with $\tanh$, whose monotonic, low-pass characteristic and bounded derivative dampen such perturbations.

In contrast, Fig. 6 shows that *ReLU* networks consistently fail to capture the solution across all architectures and hyperparameters. These results confirm that smooth activation functions, such as $\tanh$, $\sin$ are essential to scientific computing using DNNs.

B. Network Depth and Width

We vary network depth and width in a $\tanh$ network and quantify accuracy and efficiency, as shown in Fig. 7.

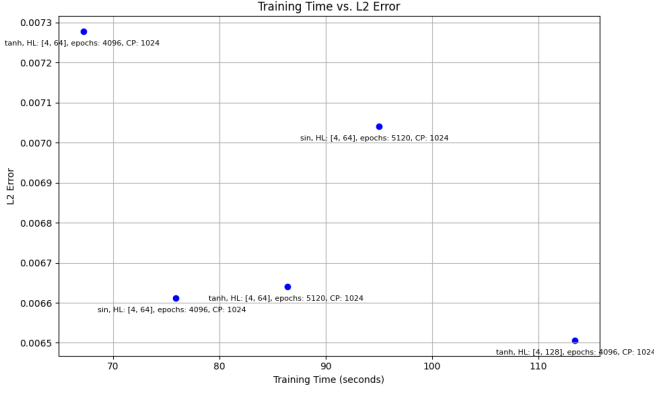


Fig. 3: Training time vs. L^2 error of sin and tanh networks

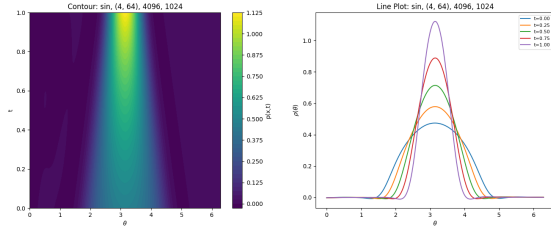


Fig. 4: Simulation profile of sin network.

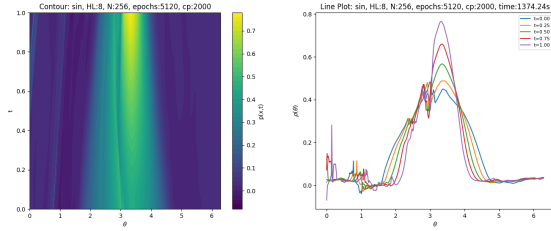


Fig. 5: Nonphysical artefacts in a sin network simulation profile.

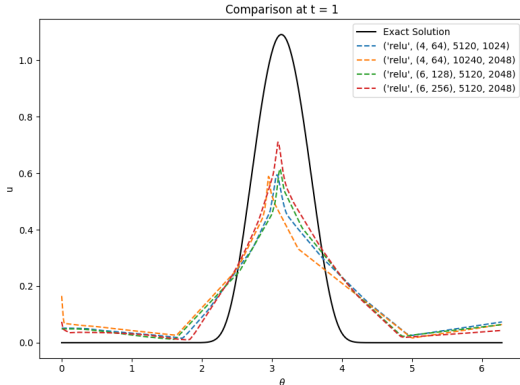


Fig. 6: Solutions of ReLU networks.

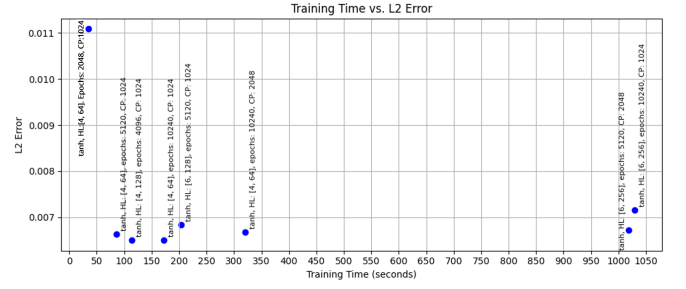


Fig. 7: Training time vs. L^2 error of tanh networks.

The configuration $\{L=4, n=64\}$, $N_r=1024$, $n_e=2048\}$ completes training in only 35 s but incurs a relatively high error of 1.11×10^{-4} . Doubling the epoch budget to $n_e = 4096$ for the same architecture yields approximately a twofold reduction in error ($\sim 7 \times 10^{-5}$) for a modest increase in training time (~ 70 s).

Increasing the width from $n=64$ to $n=128$ (with $L=4$, $N_r=1024$, $n_e=4096$) raises the training time to ~ 114 s but further reduces error down to 6.65×10^{-5} . Demonstrating that in simple PDEs, width scaling may be more effective in capturing the solution dynamics than simply extending epochs.

Epoch budgets, n_e , beyond 5120 produces diminishing returns but show significant increases in training times, indicating that the networks converged in $n_e \leq 5120$. To avoid wasted computations, it is essential to monitor convergence of the training loss during epochs along with techniques like early-stopping.

Deepening to $L=6$ and widening to $n=256$ dramatically increases training time to > 1000 s, while offering negligible error reduction. This confirms that large feed-forward architectures do not offer any benefits over smaller networks, for simple PDEs. However, for simple PDEs caution must be exercised as over-parameterisation may lead to *memorisation* rather than generalisation.

Overall, the Pareto-front of minimal training time versus minimal L^2 error is traced by shallow ($L=4$), moderately wide ($n = 64-128$) networks with epoch counts, n_e , in the range 4096–5120, giving the best balance between convergence speed and accuracy. The solutions produced by these architectures are shown in Fig. 8, in contrast to a ReLU-based plot.

C. Collocation Density

Fig. 7 shows that raising the collocation density from $N_r=1024$ to $N_r=2048$ does not produce any substantial accuracy gains unless accompanied by either prolonged training or a larger network. Even then the gains are marginal ($< 10\%$). This behaviour suggests that denser sampling alone cannot improve the solution unless the network has sufficient representational capacity and training time to exploit the extra information. Empirically, $N_r=1024$ consistently struck a good balance between most of the accuracy benefits while keeping the wall-clock cost low in typical GPU-laptop runtimes, particularly for simple PDEs.

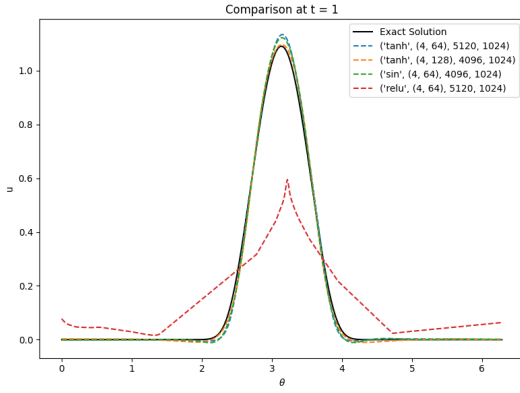


Fig. 8: Solutions of trained DNNs against reference solution.

D. Limitations: Discontinuous Initial Data

While standard feed-forward PINNs demonstrate competitive accuracy on smooth initial conditions, our experiments reveal fundamental shortcomings when the initial data contains singularities or piecewise-constant functions. We explore these shortcomings by training several architectures and activation functions on

1. Dirac-delta initial condition described in [7] as

$$\delta_\varepsilon(x-a) = \begin{cases} \frac{1}{2\varepsilon}, & \text{if } |x-a| < \varepsilon, \\ 0, & \text{otherwise,} \end{cases} \quad H(x) = \begin{cases} 1, & \text{if } x \geq 0, \\ 0, & \text{if } x < 0. \end{cases} \quad (9)$$

$$u_0(\theta) = \frac{1}{4} [\delta_\varepsilon(\theta - \frac{3\pi}{4}) + \delta_\varepsilon(\theta - \frac{5\pi}{4})] + \frac{1}{2} H(\theta - \frac{\pi}{2}) [1 - H(\theta - \frac{3\pi}{2})] \quad (10)$$

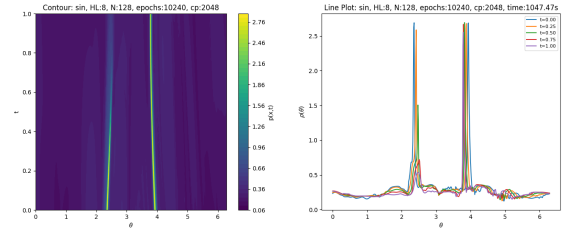
The closest solutions evolved as shown in Fig. 9.

2. Piecewise-constant initial condition described in [7] as

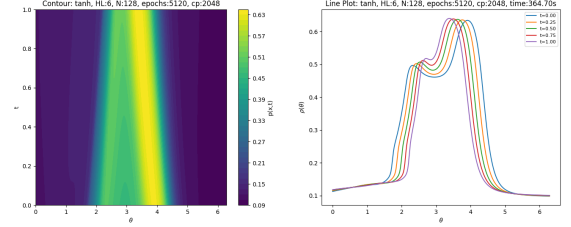
$$u_0(\theta) = \begin{cases} \frac{2}{3\pi}, & \text{if } \frac{\pi}{2} \leq \theta \leq \frac{3\pi}{2}, \\ \frac{1}{3\pi}, & \text{otherwise.} \end{cases} \quad (11)$$

The closest solutions evolved as shown in Fig. 10.

We find that all tested configurations oversmooth the sharp jumps and discontinuities present in these initial conditions. The output approximate solution looks like a smooth function, as shown in Fig. 9(b), dampening these spikes. The piecewise-constant data is approximated more accurately than the singular data, by networks using sin and *ReLU* activations, due to the oscillatory behaviour of sin and the piecewise-linear nature of ReLU. However, as shown in Fig. 10, both networks exhibit spurious oscillations near discontinuities—a Gibbs phenomenon-like artefact, arising from the difficulty of representing abrupt changes by the network.

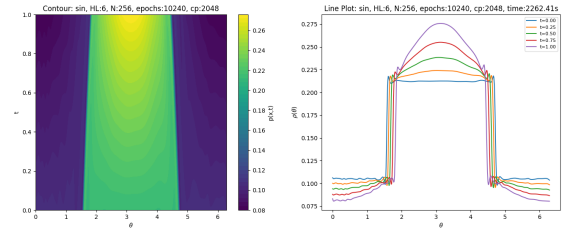


(a) sin activation

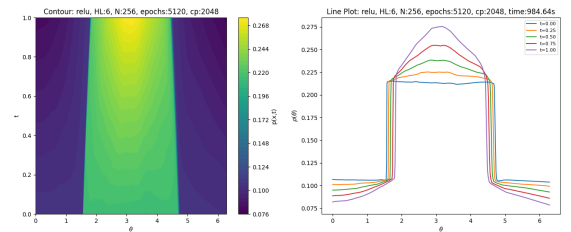


(b) tanh activation

Fig. 9: Simulation profiles of the closest solutions for initial data with singular masses.



(a) sin activation



(b) ReLU activation

Fig. 10: Simulation profiles of the closest solutions for initial data with discontinuities.

However, this is consistent with neural-network behaviour because of:

- **Smooth Activation Bias:** As discussed in Section 6.2 of [9], and theoretically analysed in [10] smooth activation functions used in DNNs for scientific computing (e.g., tanh, sin) enforce C^∞ solution spaces. This means that the resulting composite map

$$u_\Phi(\theta, t) = W^{(L+1)} \sigma(W^{(L)}(\dots \sigma(W^{(1)}[\theta, t]) \dots)) \quad (12)$$

is in C^∞ class, i.e., an infinitely differentiable function. Therefore, it cannot have a true discontinuity or delta-like spikes. Any such feature would require an unbounded derivative, which at that point would blow up.

- **Strong formulation of PDE:** A standard PINN sees the PDE in its strong (point-wise) form, $r(\theta_i, t_j)$, and then trains to minimise $MSE(r(\theta_i, t_j)) \rightarrow 0$. Automatic differentiation using auto-grad gives us the strong derivative of a smooth network even when the true solution has weak derivative or a distributional derivative like a delta function. At the location of a jump, u_Φ smoothly interpolates the discontinuity, thereby misinterpreting the physics of the discontinuity [11]. Hence, there is no large, localised penalty at that interface and the optimiser satisfies $r \approx 0$ in the MSE most easily by converging to a lower-curvature surrogate that smoothens the discontinuities.

E. Remedies for Discontinuities and Spikes

To mitigate the inherent oversmoothing of discontinuities by standard feed-forward DNNs, future studies and applications should look into any of the following techniques: weak or variational loss formulations that integrate the residual against test functions [12], thereby providing distributional derivatives and gradients at jumps. Domain decomposition by training separate subnetworks on subintervals [13] so as to localise shocks and prevent global smoothing; this would however require a priori knowledge of domain and solution structure. Enriched input embeddings, e.g., Fourier features [14] or wavelets may further enhance the network's expressivity to capture steep gradients. One emerging field is quantum machine learning, wherein Parameterised Quantum Circuits (PQCs) use quantum effects to capture nonlocal correlations [15]. Work in these techniques can extend DNNs' applicability to problems featuring discontinuities and singular measures.

V. CONCLUSION

Within this work, we have presented a systematic and comprehensive study of feed-forward DNN architectures, training styles, and their inherent limitations. By applying a PINN to a nonlocal conservation law - the identical-oscillator Kuramoto equation, and performing an extensive grid search over activation functions, network depth and width, collocation density, and epoch budgets, we identified that moderately-sized networks with four to six hidden layers of 64 to 128 neurons each, with tanh activation, prove to be the most stable and provide the most competitive accuracy, and training times.

We have empirically demonstrated the limitations of standard DNNs in approximating functions with discontinuities and singular masses due the combined effect of smooth activation function and auto-grad on strong-form PDE residual. To overcome these shortcomings we have outlined possible future directions of research with promising scope.

Our experiments and findings, thus clearly lay empirical foundations for architectural considerations in DNN models for scientific computing. Practitioners must heed these

guidelines and, where necessary, adopt specialised remedies to overcome the limitations identified. Therefore, this work effectively enriches the current body of literature on the use of DNNs for scientific and industrial applications.

REFERENCES

- [1] L. Burzawa, L. Li, X. Wang, A. Buganza-Tepole, and D. M. Umulis, "Acceleration of PDE-based biological simulation through the development of neural network metamodels," *Curr. Pathobiol. Rep.*, vol. 8, no. 4, pp. 121–131, Dec. 2020, [Online]. Available: <https://doi.org/10.1007/s40139-020-00216-8>.
- [2] M. Burger, L. Caffarelli, and P. A. Markowich, "Partial differential equation models in the socio-economic sciences," *Philos. Trans. R. Soc. A, Math. Phys. Eng. Sci.*, vol. 372, no. 2028, p. 20130406, Nov. 2014, [Online]. Available: <https://doi.org/10.1098/rsta.2013.0406>.
- [3] N. Chatterjee and U. S. Fjordholm, "Convergence of second-order, entropy stable methods for multi-dimensional conservation laws," *ESAIM: Math. Model. Numer. Anal.*, vol. 54, no. 4, pp. 1415–1428, Jul.–Aug. 2020, [Online]. Available: <https://doi.org/10.1051/m2an/2019090>.
- [4] S. Cuomo, V. S. Di Cola, F. Giampaolo, et al., "Scientific machine learning through physics-informed neural networks: where we are and what's next," *J. Sci. Comput.*, vol. 92, no. 88, 2022, [Online]. Available: <https://doi.org/10.1007/s10915-022-01939-z>.
- [5] A. Bonfanti, R. Santana, M. Ellero, and B. Gholami, "On the generalization of PINNs outside the training domain and the hyperparameters influencing it," *Neural Comput. & Applic.*, vol. 36, pp. 22677–22696, Sep. 2024, [Online]. Available: <https://doi.org/10.1007/s00521-024-10178-2>.
- [6] H. Dietert and B. Fernandez, "The mathematics of asymptotic stability in the Kuramoto model," *Proc. R. Soc. A*, vol. 474, no. 2210, p. 20180467, 2018, [Online]. Available: <http://doi.org/10.1098/rspa.2018.0467>.
- [7] N. Chatterjee and U. S. Fjordholm, "A convergent finite volume method for the Kuramoto equation and related nonlocal conservation laws," *IMA Journal of Numerical Analysis*, vol. 40, no. 1, pp. 405–421, Jan. 2020, [Online]. Available: <https://doi.org/10.1093/imanum/dry074>.
- [8] C. Chen, Y. Yang, Y. Xiang, and W. Hao, "Automatic differentiation is essential in training neural networks for solving differential equations," *J. Sci. Comput.*, vol. 104, art. no. 54, 2025, [Online]. Available: <https://doi.org/10.1007/s10915-025-02965-3>.
- [9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.
- [10] S. Wang, X. Yu, and P. Perdikaris, "When and why PINNs fail to train: A neural tangent kernel perspective," *J. Comput. Phys.*, vol. 449, p. 110768, 2022, [Online]. Available: <https://doi.org/10.1016/j.jcp.2021.110768>.
- [11] L. Liu, S. Liu, H. Xie, F. Xiong, T. Yu, M. Xiao, L. Liu, and H. Yong, "Discontinuity computing using physics-informed neural networks," *SSRN Electronic Journal*, 2022, [Online]. Available: <https://doi.org/10.2139/ssrn.4224074>.
- [12] E. Kharazmi, Z. Zhang, and G. E. M. Karniadakis, "hp-VPINNs: Variational physics-informed neural networks with domain decomposition," *Computer Methods in Applied Mechanics and Engineering*, vol. 374, p. 113547, 2021, [Online]. Available: <https://doi.org/10.1016/j.cma.2020.113547>.
- [13] A. Jagtap and G. E. M. Karniadakis, "Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations," *Communications in Computational Physics*, vol. 28, pp. 2002–2041, 2020, [Online]. Available: <https://doi.org/10.4208/cicp.OA-2020-0164>.
- [14] S. Wang, H. Wang, and P. Perdikaris, "On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 384, p. 113938, 2021, [Online]. Available: <https://doi.org/10.1016/j.cma.2021.113938>.
- [15] A. Barthe, M. Grossi, S. Vallecorsa, J. Tura, and V. Dunjko, "Parameterized quantum circuits as universal generative models for continuous multivariate distributions," arXiv preprint arXiv:2402.09848, 2024, [Online]. Available: <https://doi.org/10.48550/arXiv.2402.09848>.